# Le Mans Université

Licence Informatique *2ème année*

Module 174UP02 Rapport de projet

**Point Zéro**

Oleh DRAHOMIRETSKYI et Andrii MYKYTENKO

16 avril 2026

# Table des matières

**1 Introduction** **3**

**2 Conception** **4**

2.1 Présentation du jeu . . . . . 4

2.2 Fonctionnalités . . . . . 6

**3 Organisation du projet** **7**

3.1 Planning prévisionnel . . . . . 7

3.2 Répartition des tâches . . . . . 9

**4 Développement** **10**

4.1 Interface graphique . . . . . 10

4.2 Gestion du placement des navires . . . . . 13

4.3 Gestion de la partie . . . . . 14

4.4 Algorithmes du jeu contre ordinateur (niveaux de difficulté) . 15

**5 Résultats et conclusion** **16**

5.1 Analyse des résultats obtenus . . . . . 16

5.2 Difficultés rencontrées . . . . . 17

5.3 Conclusion et perspectives . . . . . 17

**6 Bibliographie** **19**

# 1 Introduction

Dans le cadre de ce projet, nous avons conçu et développé « Point Zéro », une adaptation numérique modernisée du célèbre jeu de la Bataille Navale. Développé en langage C à l'aide de la bibliothèque graphique SDL2, ce projet ne se contente pas de reproduire les mécaniques classiques de destruction de flotte ; il vise à transformer un jeu de plateau traditionnel en une expérience logicielle dynamique, visuelle et interactive.

L'objectif principal de ce travail est de mettre en pratique les concepts fondamentaux de la programmation impérative - manipulation de tableaux bidimensionnels, gestion de structures de données complexes et allocation dynamique - tout en relevant le défi de la programmation événementielle liée à une interface graphique. La structuration du code en modules (logic.c, render.c, main.c) a été au cœur de notre réflexion pour garantir un programme robuste et évolutif.

« Point Zéro » se distingue par plusieurs enrichissements qui modernisent le gameplay :

1. Une IA évolutive : Trois niveaux de difficulté (Facile, Normal, Expert) basés sur des algorithmes allant de l'aléatoire pur à la gestion de cartes de densité de probabilité.
2. Un système de progression : L'intégration d'une boutique virtuelle permettant de débloquer des skins (thèmes visuels) grâce à une monnaie gagnée lors des victoires, ajoutant ainsi une dimension de persistance via la gestion de fichiers de sauvegarde (records.dat).
3. Une interface immersive : Un environnement graphique soigné incluant des animations de particules en temps réel et un rendu visuel fluide.

Ce rapport détaille le cheminement de notre réflexion, de la conception logique des règles jusqu'aux choix algorithmiques avancés qui animent notre intelligence artificielle.

# 2 Conception

## 2.1 Présentation du jeu

La « Bataille navale » est un jeu de déduction et de stratégie opposant deux amiraux virtuels. Le but est simple : couler l'intégralité de la flotte adverse avant que la nôtre ne soit détruite. Le jeu se décompose en deux phases distinctes : la préparation tactique et le combat.[1]

FIGURE 1 – Exemple du jeu de bataille navale classique

**La phase de préparation : le déploiement**

Chaque joueur dispose d'une grille de 10×10 cases (identifiées par des coordonnées) et d'une flotte de navires de tailles variées.

— Le placement : les joueurs disposent leurs navires horizontalement ou verticalement. Une règle cruciale, scrupuleusement intégrée dans notre code, interdit aux navires de se chevaucher ou de se toucher, afin de laisser des espaces de navigation.
— Le secret : bien entendu, chaque joueur ignore la position des navires de son adversaire. On ne voit que sa propre grille et une grille vierge représentant le camp ennemi.

**La phase de combat : feu à volonté**

Le jeu se déroule au tour par tour. À chaque itération, un joueur annonce

une coordonnée cible (par exemple, la case à l'intersection de la ligne 3 et de la colonne 5). Trois scénarios peuvent alors se produire :

1. À l'eau (Manqué) : la case visée est vide. Le tour passe à l'adversaire. Sur l'interface, cela est marqué pour ne pas tirer deux fois au même endroit.
2. Touché : la case contient une partie d'un navire. Le joueur a le droit de rejouer immédiatement pour tenter de localiser le reste du bâtiment.
3. Coulé : toutes les cases occupées par un même navire ont été touchées. Le navire est définitivement hors de combat.

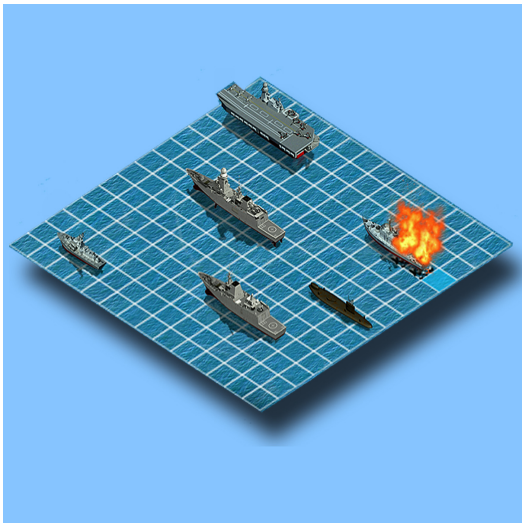

FIGURE 2 – Grille du jeu

### Condition de victoire

La partie s'achève dès qu'un joueur a réussi à envoyer par le fond les 10 navires de la flotte adverse. Dans notre version « Point Zéro », le logiciel calcule alors un score basé sur le nombre de coups utilisés, récompensant la précision et la rapidité du joueur.

## 2.2 Fonctionnalités

Le jeu Point Zéro propose un ensemble complet de fonctionnalités qui enrichissent et modernisent l'expérience classique de la Bataille Navale. Dès le lancement de l'application, le joueur est accueilli par un écran de démarrage animé (splash screen) avant d'accéder au menu principal. Ce menu centralise toutes les actions disponibles : démarrer une partie, accéder aux réglages, visiter la boutique, consulter les records ou quitter le jeu.

**Gestion de la difficulté**

Avant de lancer une partie, le joueur peut choisir parmi trois niveaux de difficulté : Facile, Normal et Expert. Ce choix influence directement le comportement de l'intelligence artificielle adverse, qui adapte sa stratégie de tir en fonction du niveau sélectionné. Cette option permet au jeu de s'adresser aussi bien aux joueurs débutants qu'aux joueurs expérimentés.

**Phase de placement des navires**

Avant chaque partie, une phase dédiée permet au joueur de configurer sa flotte. Il peut positionner manuellement chaque navire sur sa grille à l'aide de la souris, en choisissant leur orientation (horizontal ou vertical), ou recourir à un placement entièrement aléatoire généré automatiquement par le programme. La flotte est composée de 10 navires de tailles différentes : un navire de taille 4, deux de taille 3, trois de taille 2 et quatre navires de taille 1. Le système vérifie automatiquement la validité du placement en s'assurant qu'aucun navire ne se touche, ni en ligne droite ni en diagonale, conformément aux règles classiques du jeu.

**Déroulement de la partie**

La partie se joue en alternance entre le joueur et l'ordinateur. Le joueur clique sur la grille adverse pour effectuer ses tirs, et les cellules changent de couleur selon le résultat : tir dans l'eau, touché ou coulé. Lorsqu'un navire est entièrement détruit, les cases adjacentes sont automatiquement marquées comme sécurisées, évitant ainsi des tirs inutiles. La partie se termine lorsque l'un des deux adversaires a coulé l'intégralité de la flotte ennemie.

**Système économique et boutique**

Point Zéro introduit un système économique inédit dans la Bataille Navale : à la fin de chaque partie gagnée, le joueur reçoit des pièces virtuelles.

Ces pièces peuvent être dépensées dans la boutique du jeu pour débloquer de nouveaux skins visuels pour les grilles. Huit thèmes sont disponibles :

| Thème | Prix(pièces) |
|---|---|
| Océan | gratuit |
| Nuit | gratuit |
| Arctique | 150 |
| Rouge | 200 |
| Or | 300 |
| Forêt | 400 |
| Plasma | 450 |
| Glace | 500 |

Table 1 – Thèmes et leurs prix

Ce mécanisme de progression encourage le joueur à rejouer et à s'améliorer.

**Suivi des statistiques**

Le jeu enregistre automatiquement les performances du joueur dans un fichier de sauvegarde (records.dat). L'écran des records affiche le nombre total de victoires et de défaites, le taux de réussite en pourcentage ainsi que le score de la meilleure partie. Ces données persistent entre les sessions de jeu, permettant au joueur de suivre sa progression sur le long terme.

# 3 Organisation du projet

## 3.1 Planning prévisionnel

Le développement du projet Point Zéro a été planifié et organisé en quatre phases principales, réparties sur une période allant du 12 janvier 2026 au 12 avril 2026, sous la responsabilité du chef de projet Oleh Drahomiretskyi. Chaque phase regroupe un ensemble de tâches cohérentes, avec des dates de début et de fin précises, permettant de suivre l'avancement du projet de manière structurée.

**Phase 1 - Initialisation (12.01 - 01.02)**

La première phase couvre la période du 12 janvier au 1er février 2026 et constitue le socle du projet. Elle comprend deux tâches principales. La tâche 1.1 porte sur l'analyse des règles du jeu et la définition des fonctionnalités : cette étape a permis de formaliser les mécaniques du jeu (placement des navires, système de tir, conditions de victoire, niveaux de difficulté) et de dresser la liste complète des fonctionnalités à implémenter. La tâche 1.2 concerne la conception de l'architecture du programme : définition des modules (logic, render, main), choix des structures de données et organisation des fichiers sources. Cette phase s'étend jusqu'à début février, laissant le temps nécessaire à une réflexion approfondie avant d'entamer le développement.

**Phase 2 - Création des managers (02.02 - 22.02)**

La deuxième phase se déroule du 2 au 22 février 2026 et est entièrement consacrée au développement des composants fondamentaux du jeu. Elle se divise en quatre tâches. La tâche 2.1 traite de la gestion des grilles : implémentation de la structure GameBoard, initialisation, affichage et gestion des états de chaque cellule. La tâche 2.2 couvre la gestion des bateaux : logique de placement valide, vérification des contraintes d'espacement et placement aléatoire. La tâche 2.3 correspond à la gestion des joueurs : mise en place des entités joueur et ordinateur, ainsi que les structures associées (CPUState, GameStats). Enfin, la tâche 2.4 regroupe les tests unitaires des managers, permettant de valider le bon fonctionnement de chaque composant avant de passer à la phase suivante.

**Phase 3 - Codage et test (02.03 - 22.03)**

La troisième phase, du 2 au 22 mars 2026, constitue le cœur du développement. La tâche 3.1 porte sur l'implémentation du placement des bateaux sur la grille avec interaction souris. La tâche 3.2 couvre l'implémentation du système de tir, elle-même subdivisée en deux sous-tâches : 3.2.1 pour la gestion des tours de jeu (alternance joueur/IA) et 3.2.2 pour l'affichage de la grille et des résultats des tirs en temps réel. La tâche 3.3 regroupe les tests fonctionnels du jeu complet, suivis de la tâche 3.3.1 dédiée à la correction des bugs et à la validation finale de l'ensemble des fonctionnalités.

**Phase 4 - Livraison (23.03 – 12.04)**

La quatrième et dernière phase s'étend du 23 mars au 12 avril 2026. Elle comprend la tâche 4.1 de rédaction du rapport de projet et la tâche 4.2 de préparation de la soutenance. Les tâches 4.3 et 4.4, prévues dans le

planning, constituent une réserve permettant de gérer d'éventuels imprévus ou ajustements de dernière minute avant la date limite du 17 avril 2026.[2]

## 3.2 Répartition des tâches

Le projet a été développé en binôme par Oleh Drahomiretskyi et Andrii Mykytenko. Dès le début du projet, nous avons défini une répartition claire des responsabilités afin de pouvoir travailler en parallèle de manière efficace, tout en maintenant une cohérence globale grâce à des points de synchronisation réguliers.

**Oleh Drahomiretskyi — Logique du jeu et intelligence artificielle**

Oleh a pris en charge l'intégralité de la couche logique du projet, implémentée dans les fichiers logic.c et logic.h. Ses responsabilités comprenaient la conception et l'implémentation des algorithmes de placement des navires (manuel et aléatoire), la validation des positions selon les règles du jeu (absence de contact entre bateaux), la gestion des tirs et la détection des navires coulés avec marquage automatique des cases adjacentes en zone sécurisée.

Il a également développé l'algorithme d'intelligence artificielle adversaire, qui s'appuie sur un système de pile de cibles prioritaires. Lorsqu'un tir touche un navire, l'IA pousse les cases voisines sur la pile et, dès qu'un alignement est détecté (vertical ou horizontal), concentre ses tirs dans cet axe jusqu'à couler le navire. Cet algorithme est paramétré selon trois niveaux de difficulté qui influencent la précision et l'agressivité des tirs de l'ordinateur. Enfin, Oleh a géré la persistance des données avec les fonctions LoadStats et SaveStats, qui permettent de sauvegarder et de recharger les statistiques du joueur (victoires, défaites, pièces, skins débloqués) entre les sessions via le fichier records.dat.

**Andrii Mykytenko — Interface graphique et boucle principale**

Andrii a été responsable de tout le rendu visuel du jeu, implémenté dans les fichiers render.c, render.h et main.c. Il a conçu et développé l'affichage des grilles de jeu, le système de coloration dynamique des cellules selon leur état, ainsi que l'ensemble des écrans de l'application (splash screen, menu principal, paramètres, boutique, placement, jeu, fin de partie, records).

Il a également créé le composant de bouton interactif avec des effets vi-

suels avancés (dégradé de couleur, ombre portée, animation au survol), et a conçu les huit thèmes visuels (skins) appliqués à l'ensemble de l'interface. L'animation de particules flottantes en arrière-plan, qui s'affiche sur tous les écrans, a également été réalisée par Andrii.

Dans main.c, il a implémenté la boucle principale du jeu avec la gestion complète des événements SDL2 (clics souris, mouvements, touches clavier) et les transitions entre les différents modes de jeu selon l'état courant.

**Travail collaboratif**

Les deux membres ont collaboré sur la définition des structures de données partagées dans common.h, sur l'intégration entre logique et rendu, et sur les tests finaux.

# 4 Développement

## 4.1 Interface graphique

L'interface graphique de Point Zéro a été entièrement développée en langage C à l'aide de la bibliothèque SDL2 (Simple DirectMedia Layer)[3], complétée par l'extension SDL2_ttf pour le rendu typographique. Ce choix technologique garantit des performances élevées, une gestion fluide des événements en temps réel et une compatibilité multiplateforme. La fenêtre de jeu s'ouvre en plein écran dans une résolution fixe de 1280 × 800 pixels.

**Architecture des écrans**

L'interface est organisée autour d'une machine à états dont les modes sont définis dans l'énumération GameMode. Chaque mode correspond à un écran distinct : l'écran de démarrage animé (MODE_SPLASH), le menu principal (MODE_MAINMENU), les paramètres (MODE_SETTINGS), la boutique (MODE_SHOP), la phase de placement (MODE_PLACEMENT), la partie en cours (MODE_GAME), l'écran de fin de partie (MODE_GAMEOVER) et le tableau des records (MODE_RECORDS). La boucle principale du jeu gère les transitions entre ces états en fonction des actions de l'utilisateur.

FIGURE 3 – Ecran principal

**Rendu des grilles de jeu**

Les deux grilles de $10 \times 10$ cases sont affichées côte à côte dans la fenêtre. Chaque cellule mesure $50 \times 50$ pixels. La grille du joueur est positionnée à gauche (offset horizontal de 60 px) et celle de l'adversaire à droite (offset de 670 px), toutes deux à une hauteur verticale commune de 150 px. Les cellules sont colorées dynamiquement selon leur état : vide, navire présent, tir manqué, tir touché, navire coulé ou zone adjacente sécurisée. Cette codification visuelle permet au joueur de lire l'état de la partie en un coup d'œil.

**Système de skins**

L'un des aspects visuels les plus distinctifs du jeu est son système de thèmes. Huit skins sont disponibles, chacun définissant une palette de couleurs complète comprenant la couleur de fond, la couleur de l'eau, la grille, les navires, les tirs, les cases coulées et les zones sécurisées. Les thèmes vont de l'océan bleu nuit classique aux ambiances plus originales comme l'Arctique (tons gris-bleu glacés), le Plasma (cyan néon sur fond sombre) ou l'Or (tons dorés et bordeaux). Le skin actif est appliqué instantanément à l'ensemble de l'interface dès sa sélection.

FIGURE 4 – Thèmes dans la boutique

### Composants d'interface : boutons

Les boutons interactifs ont été conçus avec un soin particulier apporté aux détails visuels. Chaque bouton présente un dégradé de luminosité vertical (plus clair en haut, plus sombre en bas), une bordure biseautée avec un côté lumineux et un côté ombré pour un effet de relief, et une ombre portée décalée pour simuler une légère élévation. Au survol de la souris, un effet de surbrillance accentue la couleur du bouton et un halo lumineux extérieur (glow) apparaît progressivement, renforçant le retour visuel pour l'utilisateur.

### Animation de particules

Un système d'animation de particules flottantes constitue le fond vivant de tous les écrans du jeu. Quatre-vingt-dix particules se déplacent en continu dans toutes les directions à vitesse variable, en rebouclant sur les bords de la fenêtre. Lorsque deux particules se trouvent à moins de 85 pixels l'une de l'autre, une ligne translucide les relie, créant un effet de réseau neuronal ou de constellation en mouvement. Cette animation est légère en ressources, entièrement calculée par le CPU, et contribue significativement à l'atmosphère

moderne et dynamique du jeu.

**Typographie et lisibilité**

L'ensemble des textes du jeu est rendu avec une police TrueType personnalisée (font.ttf)[4] chargée via SDL2_ttf. Les textes importants (titres, étiquettes de boutons) bénéficient d'une ombre portée réalisée en affichant le texte deux fois avec un léger décalage, améliorant la lisibilité sur tous les fonds colorés. Les couleurs de texte sont également adaptées à chaque skin pour garantir un contraste suffisant en toutes circonstances.

## 4.2 Gestion du placement des navires

La phase de placement repose sur un schéma de flotte rigoureux défini par la constante SCHEMA[10]. La logique implémentée garantit le respect des règles traditionnelles de la Bataille Navale à travers deux mécanismes principaux :

**Algorithme de vérification et contraintes**

La fonction CanPlaceShip est le cœur du système de validation. Elle vérifie deux conditions critiques avant toute insertion en mémoire :

— Les limites de la grille : elle s'assure que le navire, selon son orientation (horizontale ou verticale), ne dépasse pas les dimensions GRID_SIZE.
— La zone d'exclusion : contrairement à une vérification de simple chevauchement, l'algorithme parcourt un voisinage de 8 cases (boucles dr et dc de -1 à 1) autour de chaque segment du futur navire. Cela garantit qu'aucun bateau ne touche un autre, même par la diagonale.

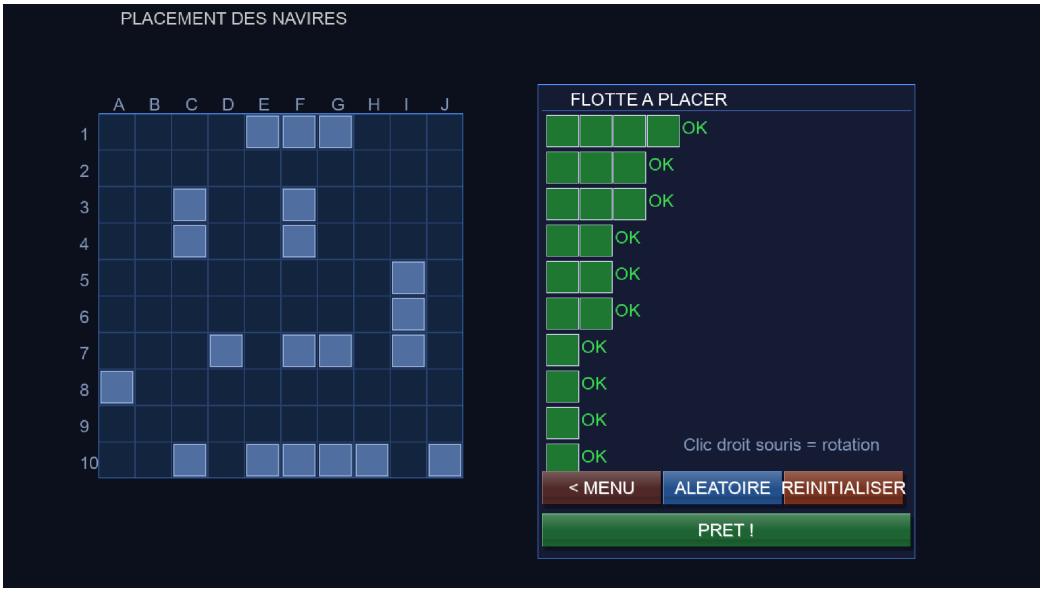


FIGURE 5 – Placement de la flotte

**Placement aléatoire et robustesse**

Pour la fonction RandomPlaceFull, nous avons utilisé une approche par "essai et erreur" (Brute-force). Le programme tire des coordonnées et une orientation de manière aléatoire jusqu'à ce que CanPlaceShip renvoie true. Grâce à la faible densité des navires sur une grille de $10 \times 10$, cette méthode est quasi instantanée et évite toute complexité inutile.

## 4.3 Gestion de la partie

La gestion du déroulement de la partie s'appuie sur une structure de données GameBoard et une boucle de jeu gérant l'alternance des tours.

**Mécanique des tirs et retour visuel**

Chaque tir modifie l'état d'une cellule selon une énumération précise (CELL_HIT, CELL_MISS, etc.). Un aspect important de notre implémentation est la fonction CheckSunkShips. Lorsqu'un navire est touché, le programme vérifie par propagation (fonction ShipIsSunk) si toutes les parties du navire ont été atteintes.

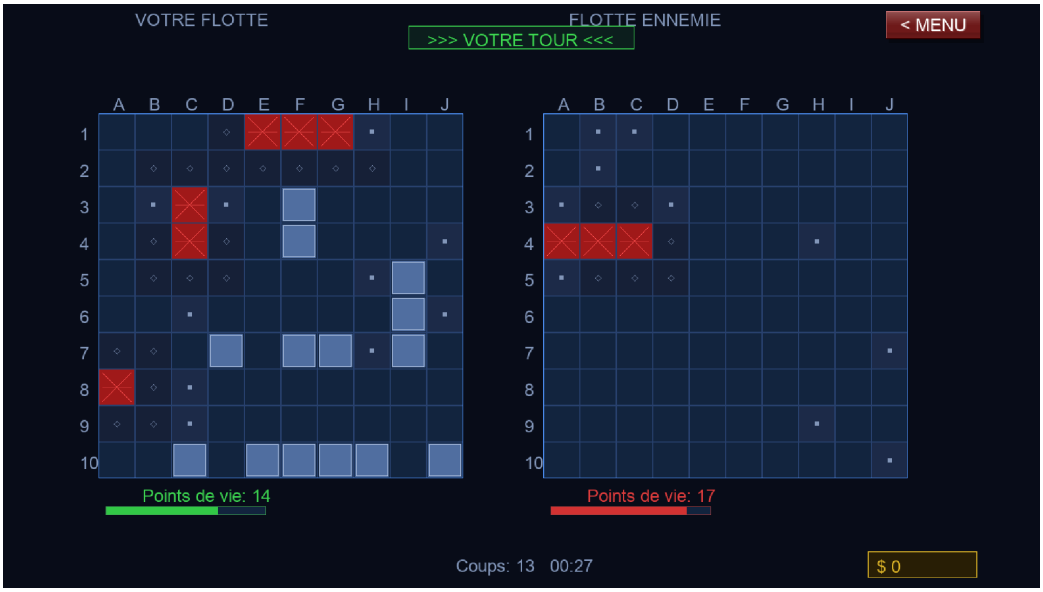


FIGURE 6 – Partie en cours

**Automatisation des zones sécurisées**

Pour améliorer l'expérience utilisateur, nous avons implémenté la fonction MarkSunk. Une fois un navire coulé :

1. Il est marqué visuellement avec l'état CELL_SUNK.
2. Toutes les cases vides adjacentes sont automatiquement passées en CELL_NEAR_SUNK. Cela permet au joueur (et à l'IA) de ne pas perdre de tours à tirer dans des zones où il est mathématiquement impossible de trouver un autre navire.

## 4.4 Algorithmes du jeu contre ordinateur (niveaux de difficulté)

L'intelligence artificielle est divisée en trois niveaux de difficulté, exploitant différentes stratégies de recherche et de ciblage.

**Mode Facile : aléatoire pur**

L'IA utilise la fonction RandomUnshot qui sélectionne une cible au hasard

parmi toutes les cases n'ayant pas encore reçu de tir.

**Mode Normal : algorithme de "Hunt & Target"**

Ce mode utilise une pile de cibles prioritaires (cpu->stack).

— Phase de chasse (Hunt) : l'IA tire au hasard jusqu'à toucher un navire.
— Phase de ciblage (Target) : dès qu'un navire est touché, les cases adjacentes sont ajoutées à la pile. La fonction PushTargets est optimisée : si deux touches consécutives révèlent une ligne (verticale ou horizontale), l'IA n'ajoute que les cases situées dans l'alignement détecté, ignorant les autres directions pour gagner en efficacité.[5]

**Mode Expert : densité de probabilité et Parité**

Le niveau Expert remplace la chasse aléatoire par la fonction BestHuntCell :

— Carte de densité : l'algorithme simule le placement de tous les navires restants (cpu->remain) sur toutes les positions valides de la grille. Chaque case reçoit un score basé sur le nombre de combinaisons possibles. L'IA tire sur la case ayant la plus haute probabilité de contenir un navire. [5]
— Stratégie de parité : pour optimiser la recherche des petits navires, l'algorithme privilégie un motif en "damier" (r+c % 2), ce qui permet de couvrir la grille deux fois plus vite.

# 5 Résultats et conclusion

Le projet Point Zéro a abouti à la création d'une version numérique complète et modernisée de la Bataille Navale. Le développement en langage C, appuyé par la bibliothèque SDL2, nous a permis d'atteindre les objectifs fixés initialement tout en relevant des défis techniques stimulants.

## 5.1 Analyse des résultats obtenus

Le programme final remplit l'ensemble des fonctionnalités prévues dans le cahier des charges :

— Stabilité et performance : l'application est fluide, tournant à une réso-

lution de $1280 \times 800$ pixels sans ralentissement, même avec l'animation de fond de particules calculée en temps réel.

— Intelligence Artificielle : le système de pile de cibles prioritaires offre un défi réel au joueur. L'algorithme parvient efficacement à traquer et couler les navires une fois le premier contact établi, simulant un comportement humain crédible.
— Expérience utilisateur : l'interface graphique est intuitive. Le système de skins et la boutique virtuelle ajoutent une dimension de progression absente du jeu traditionnel, ce qui augmente considérablement la durée de vie du logiciel.
— Robustesse : la gestion des erreurs, notamment pour la validation du placement des navires et la sauvegarde des données dans records.dat, garantit une expérience sans crash et une persistance fiable des statistiques.

## 5.2 Difficultés rencontrées

Le principal défi a résidé dans l'intégration de la logique de jeu avec le rendu graphique en temps réel. La synchronisation des tours entre le joueur et l'ordinateur, ainsi que la gestion de la machine à états pour les transitions entre les menus, a nécessité une structuration rigoureuse du code dans main.c. De plus, l'algorithme de placement aléatoire a dû être optimisé pour éviter les boucles infinies tout en respectant les contraintes strictes d'espacement entre les navires.

## 5.3 Conclusion et perspectives

Ce projet nous a permis de consolider nos compétences en programmation C, notamment sur la manipulation de structures de données complexes et la gestion de bibliothèques externes. Nous avons appris à organiser un travail collaboratif efficace en binôme, en séparant clairement les responsabilités tout en maintenant une communication constante sur les interfaces communes.

Pour la suite, plusieurs pistes d'amélioration pourraient être envisagées :

— Mode Multijoueur en ligne : Permettre à deux utilisateurs de s'affronter via une connexion réseau (sockets).

— Effets Sonores : Ajouter une ambiance sonore et des bruitages pour les tirs et les explosions afin de renforcer l'immersion.
— Refonte esthétique et réalisme : une évolution majeure consisterait à remplacer le rendu actuel par des graphismes plus réalistes. L'ajout de skins utilisant des textures haute définition, des modèles de navires détaillés et des effets d'eau animés par des shaders permettrait de renforcer l'immersion visuelle.

FIGURE 7 – Exemple de jeu esthétiquement plus développé

En conclusion, Point Zéro représente une synthèse réussie entre un classique du jeu de société et une implémentation logicielle moderne, offrant une base solide pour d'éventuels développements futurs.

# 6 Bibliographie

## Références


[1] https://fr.wikipedia.org/wiki/Bataille_navale_(jeu)

[2] https://docs.google.com/spreadsheets/d/1IqbCmA31hAxSB0njCKjtLSsNYLb4gT-OUikucNKHJD8/edit?usp=sharing

[3] https://zestedesavoir.com/tutoriels/1014/utiliser-la-sdl-en-langage-c/

[4] https://wiki.libsdl.org/SDL2_ttf/FrontPage

[5] https://www.actonscholars.org/wp-content/uploads/2020/08/06-Optimal_Strategies_on_Attack_and_Defense-Chanbin-Park-200902.pdf